



Escalonamento de CPU

Universidade Estadual de Mato Grosso do Sul
Bacharelado em Ciência da Computação
Sistemas Operacionais
Prof. Fabrício Sérgio de Paula

Tópicos

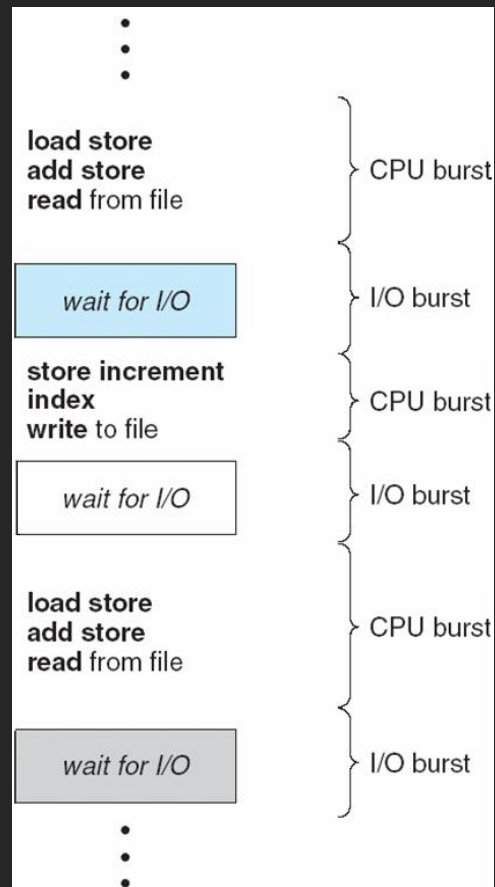
- Conceitos
- Critérios de escalonamento
- Algoritmos de escalonamento
- Escalonamento de threads
- Escalonamento em multiprocessadores
- Exemplos de sistemas operacionais
- Avaliação de algoritmos

Conceitos

- Objetivo da multiprogramação: maximizar uso da CPU
 - Diversos processos em memória
 - Sempre que um deixa a CPU (em geral E/S), outro ganha
 - O escalonamento é fundamental
 - Praticamente todos recursos são escalonados (não só CPU)
 - O escalonamento é parte central do projeto de um SO
- Execução de processo composta de:
 - Picos de CPU (CPU bursts): execução intensiva de instruções
 - Picos de E/S (I/O bursts): espera por operação de E/S

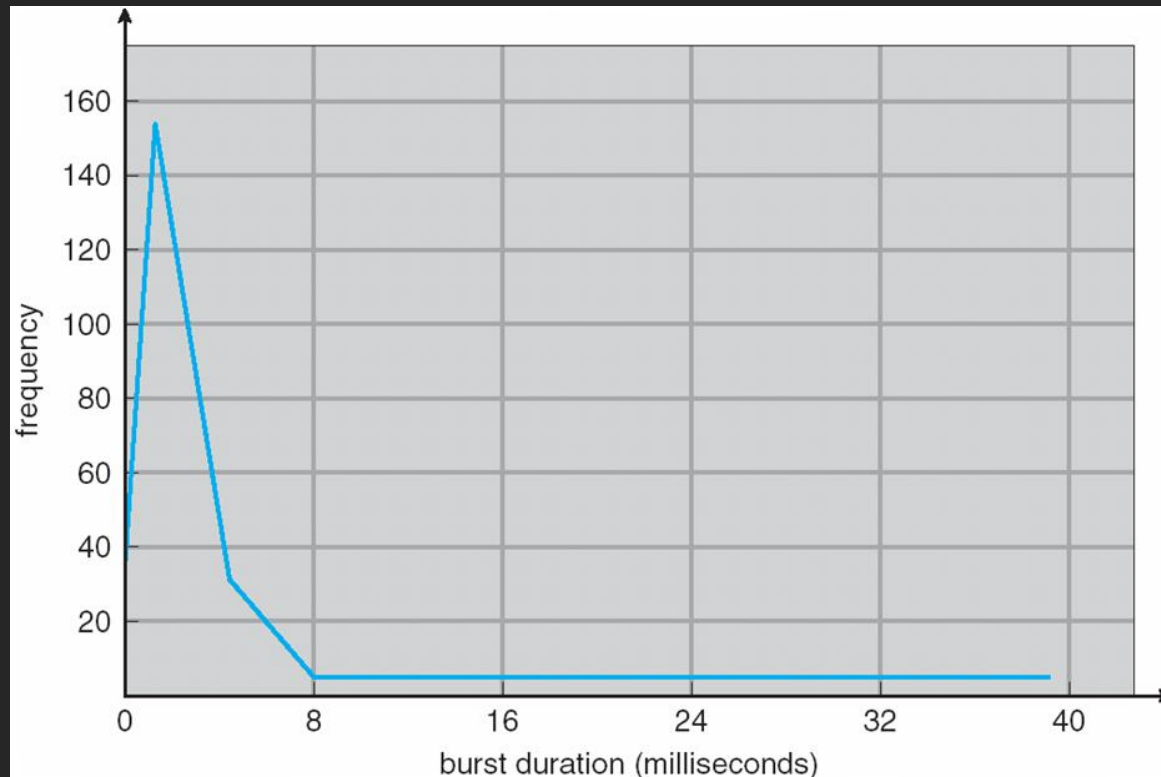
Conceitos

- Picos de CPU e de E/S de um processo:



Conceitos

- Histograma de tempo dos picos de CPU:
 - Poucos picos ultrapassam 8ms



Conceitos

- Escalonador de CPU (curto prazo): seleciona processo na fila de prontos para executar na CPU
 - Várias opções de algoritmos para essa seleção
- Preempção: interrupção da execução de um processo
 - Retira o processo da CPU de maneira “forçada”
- Escalonamento pode ser
 - Sem preempção: processo deixa a CPU espontaneamente (terminou pico de CPU)
 - Com preempção: processo é retirado da CPU dentro de pico de CPU

Conceitos

- Escalonamento pode ocorrer quando um processo:
 1. Passa do estado *em execução* para *em espera*
 2. Passa do estado *em execução* para *pronto*
 3. Passa do estado *em espera* para *pronto*
 4. Termina: passa para o estado *terminado*
- Se escalonador é invocado apenas nas circunstâncias 1 e 4, o escalonamento é sem preempção
 - Caso contrário, escalonamento é com preempção

Conceitos

- Escalonador de CPU sem preempção
 - Processo pode usar a CPU por longos períodos
 - Inviabiliza tempo compartilhado
- Escalonador de CPU com preempção
 - Requer hardware especial: timer
 - Projeto mais complexo
 - Problema: como retirar um processo da CPU se o kernel está tratando de suas atividades?
 - Pode haver inconsistências, perdas de dados
 - Alguns sistemas só fazem preempção após final de chamada ao sistema ou bloco de E/S

Conceitos

- Interrupções devem ser tratadas assim que chegam
 - Ocorrem a qualquer momento
 - Seções de código do kernel afetadas por interrupções: proteção contra uso simultâneo
 - Interrupções desabilitadas no início e habilitadas no final
 - Descartar interrupções: perda/sobrescrita de dados
 - Desabilitação não deve ocorrer frequentemente e código deve ser bem pequeno

Conceitos

- Despachante: módulo que passa a CPU ao processo escolhido pelo escalonador
 - Tarefas:
 - Trocar o contexto
 - Colocar a execução em modo usuário
 - Desviar a execução para o local apropriado do processo escolhido
- Latência de despacho: tempo gasto para o despachante operar
 - Despachante deve ser o mais rápido possível: invocado a cada troca de processo

Critérios de escalonamento

- Existem diferentes algoritmos de escalonamento
- Critérios de escalonamento permitem comparação para eleger um bom escalonador
 - Melhor escalonador: depende da situação onde vai ser usado
 - Principais critérios: utilização da CPU, throughput, tempo de turnaround, tempo de espera e tempo de resposta

Critérios de escalonamento

- Utilização de CPU: varia de 0 a 100%
 - Indica se CPU está ociosa ou sobrecarregada
 - Em uma situação real deve variar de 40% a 90%
- Throughput: quantidade de processos executados em unidade de tempo (por segundo, hora, etc.)
 - Ex.: 100 processos/segundo
- Tempo de turnaround: tempo que um processo leva, desde a submissão até o término (inclui E/S, esperas, etc.)
- Tempo de espera: tempo total que um processo gasta esperando apenas na fila de prontos
- Tempo de resposta: intervalo entre uma solicitação ao processo e a primeira resposta do processo

Critérios de escalonamento

- Objetivos de um escalonador:
 - Maximizar: utilização da CPU e throughput
 - Minimizar: tempos de turnaround, espera e resposta
- Em muitos casos, deseja-se maximizar ou minimizar valor médio
 - Em alguns casos: valores máximos/mínimos
 - Tempo de resposta: minimizar variância é melhor que média

Algoritmos de escalonamento

- FCFS (first come, first served)
- SJF (shortest job first)
 - SRTF (shortest remaining time first)
- Prioridade
- RR (round-robin)
- Filas multiníveis
- Filas multiníveis com retroalimentação

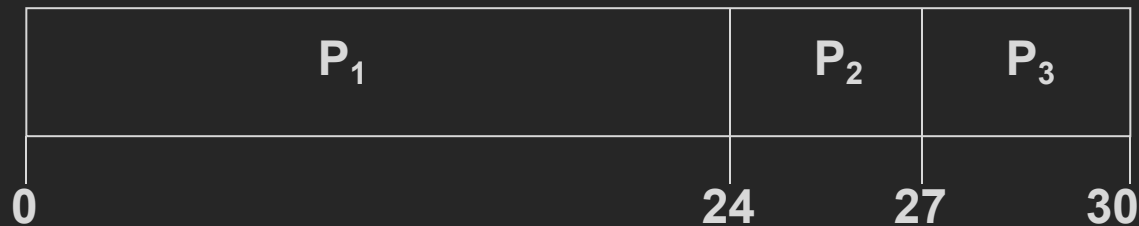
Algoritmos de escalonamento

- Algoritmo First-come, first-served (FCFS):
 - Algoritmo de escalonamento mais simples
 - Primeiro processo a entrar na fila de prontos é primeiro a ir para CPU
 - Sem preempção
 - Processo só sai da CPU espontaneamente
 - Inadequado para sistemas de tempo compartilhado
 - Implementação simples: fila de prontos FIFO

Algoritmos de escalonamento

<u>Processo</u>	<u>Pico de CPU</u>
P1	24ms
P2	3ms
P3	3ms

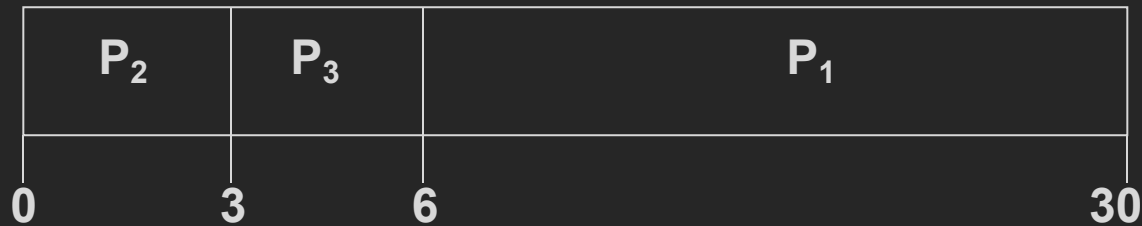
- Ordem de chegada: P1 , P2 , P3, todos no instante 0. O diagrama de Gantt para o escalonamento será:



- Tempo de espera para P1 = 0; P2 = 24; P3 = 27
- Tempo de espera médio: $(0 + 24 + 27)/3 = 17\text{ms}$
- Utilização da CPU? Tempo de turnaround?

Algoritmos de escalonamento

- Se ordem de chegada fosse P2 , P3 , P1, o diagrama seria:



- Tempo de espera para P1 = 6; P2 = 0; P3 = 3
- Tempo de espera médio: $(6 + 0 + 3)/3 = 3\text{ms}$
- Muito melhor do que no caso anterior
- Outro problema: efeito comboio
 - 1 processo CPU-bound + n I/O-bound
 - n processos fazem E/S e têm que aguardar o CPU-bound
 - Isso se repete durante toda a execução

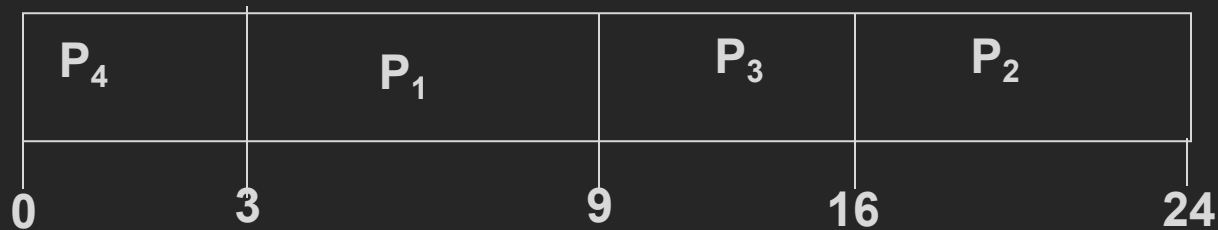
Algoritmos de escalonamento

- Algoritmo Shortest Job First (SJF):
 - Escolhe processo com menor pico de CPU
 - Não tem preempção
 - Variação com preempção: Shortest Remaining Time First (SRTF)
 - Preempção ocorre quando chega processo com pico de CPU menor que o restante do atual
- SJF é ótimo em relação ao tempo de espera médio
 - Garante menor tempo de espera médio

Algoritmos de escalonamento

<u>Processo</u>	<u>Pico de CPU</u>
P1	6ms
P2	8ms
P3	7ms
P4	3ms

- Escalonamento:



- Tempo de espera para P1 = 3; P2 = 16; P3 = 9; P4 = 0
- Tempo de espera médio: $(3+16+9+0)/4 = 7\text{ms}$

Algoritmos de escalonamento

- Algoritmo SRTF (SJF preemptivo):

<u>Processo</u>	<u>Chegada</u>	<u>Pico de CPU</u>
P1	0ms	8ms
P2	1ms	4ms
P3	2ms	9ms
P4	3ms	5ms

- Escalonamento:



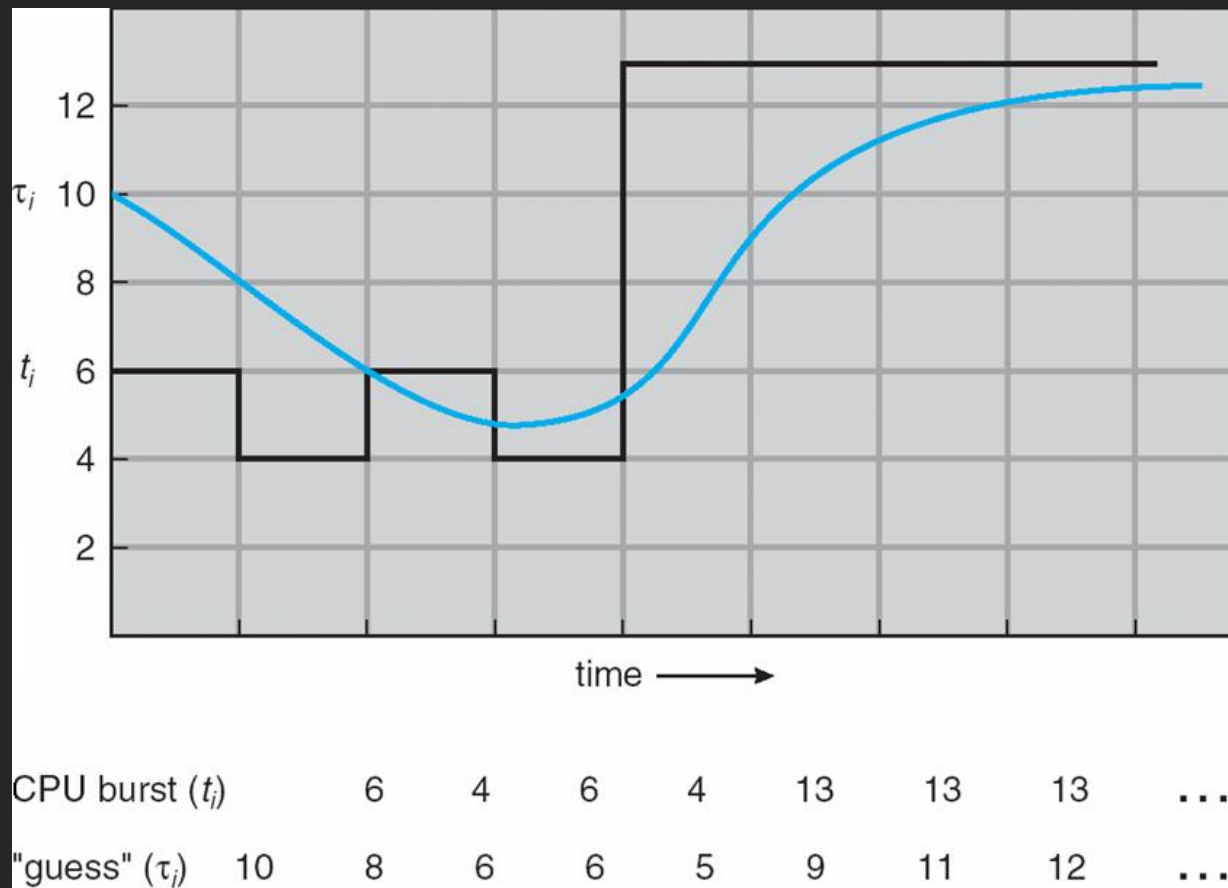
- Tempo de espera médio = $[(10 - 1) + (1 - 1) + (17 - 2) + (5 - 3)] / 4 = 6,5 \text{ ms}$

Algoritmos de escalonamento

- Dificuldades com SJF:
 - Como saber tamanho dos próximos picos de CPU?
 - Usuário informa
 - Pode-se estimar através de história de picos anteriores
 - Estimativa através de média exponencial
 - τ_{n+1} = duração estimada do próximo pico de CPU
 - τ_n = história dos picos passados
 - t_n = duração real do enésimo burst de CPU
 - Define-se: $\tau_{n+1} = \alpha t_n + (1 - \alpha) \tau_n$, onde $0 \leq \alpha \leq 1$
 - SJF não é adequado como escalonador de curto prazo

Algoritmos de escalonamento

- Estimativa através da média exponencial:



Algoritmos de escalonamento

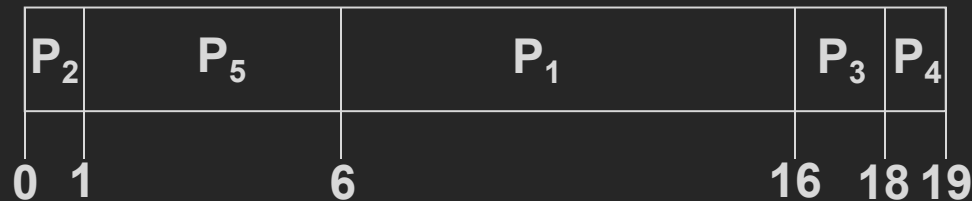
- Algoritmo por prioridade:
 - Número de prioridade é associado a cada processo
 - CPU é alocada para o processo com maior prioridade
 - Processos de mesma prioridade: FCFS
- Convenção
 - Números menores: prioridades mais altas
 - Números maiores: prioridades mais baixas

Algoritmos de escalonamento

Tempo de chegada: 0 para todos os processos

<u>Processo</u>	<u>Pico de CPU</u>	<u>Prioridade</u>
P1	10ms	3
P2	1ms	1
P3	2ms	4
P4	1ms	5
P5	5ms	2

- Escalonamento:



- Tempo de espera médio = 8,2 ms

Algoritmos de escalonamento

- Escalonamento pode ser
 - Não-preemptivo
 - Preemptivo: se chega um de prioridade mais alta, escalonador troca processo
- Possibilidade de estagnação/inanição/*starvation*
 - Processo de baixa prioridade pode nunca ser escolhido
 - Solução: envelhecimento (aging)
 - Aumenta a prioridade dos processos que estão em espera há muito tempo

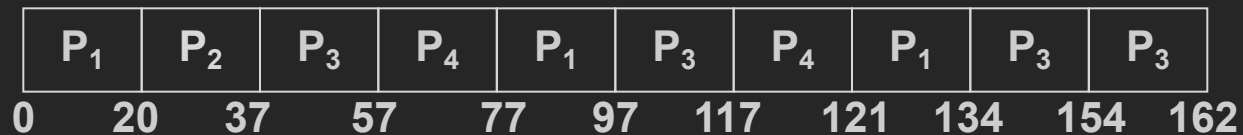
Algoritmos de escalonamento

- Algoritmo Round-robin (RR):
 - Processos são atendidos na ordem FIFO
 - Cada processo fica, no máximo, q unidades de tempo na CPU
 - Quando deixa CPU, processo vai pro final da fila de prontos
 - q é chamado de quantum: 10-100ms
 - Para n processos e quantum q
 - Cada processo recebe $1/n$ do tempo de CPU
 - Em , no máximo, q unidades de tempo cada vez
 - Nenhum processo espera mais que $(n-1)*q$ na fila de prontos

Algoritmos de escalonamento

<u>Processo</u>	<u>Pico de CPU</u> (q = 20ms)
P1	53
P2	17
P3	68
P4	24

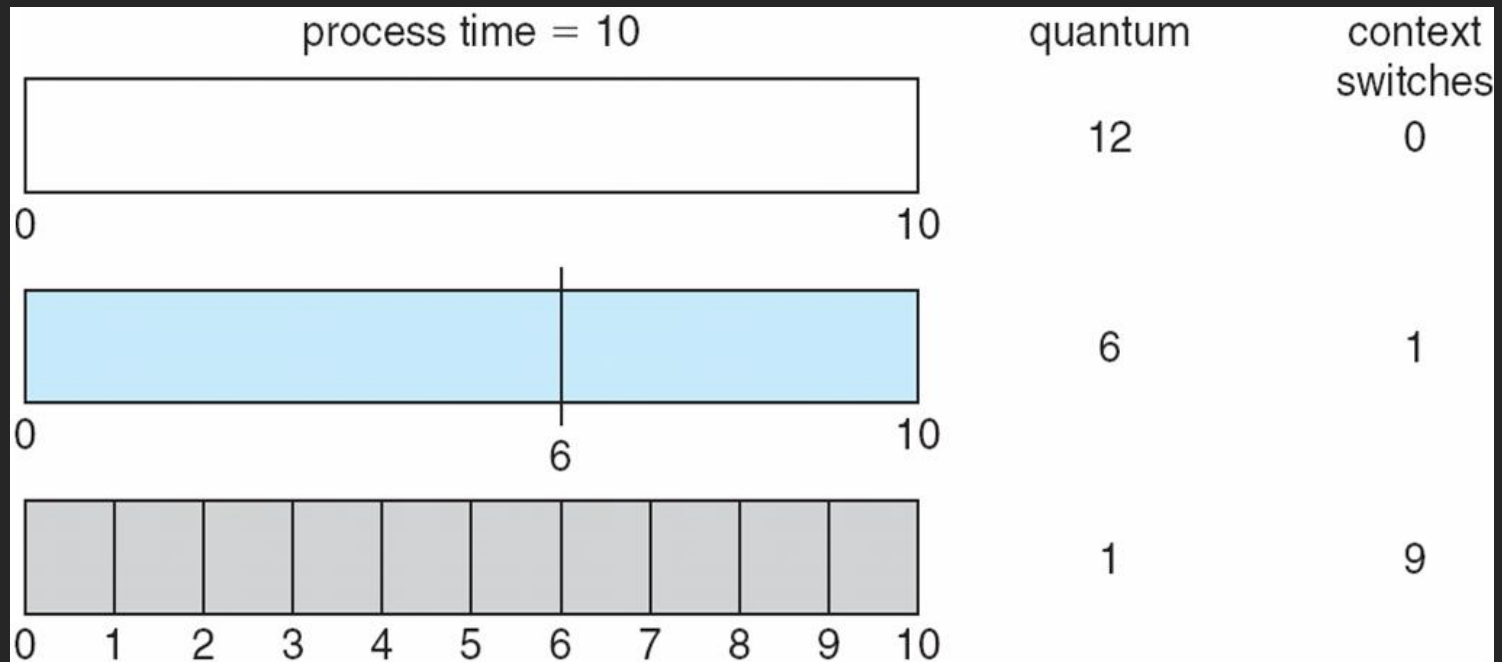
- O diagrama de Gantt é:



- Turnaround médio tende a ser maior que do SJF
 - Mas tempo de resposta menor

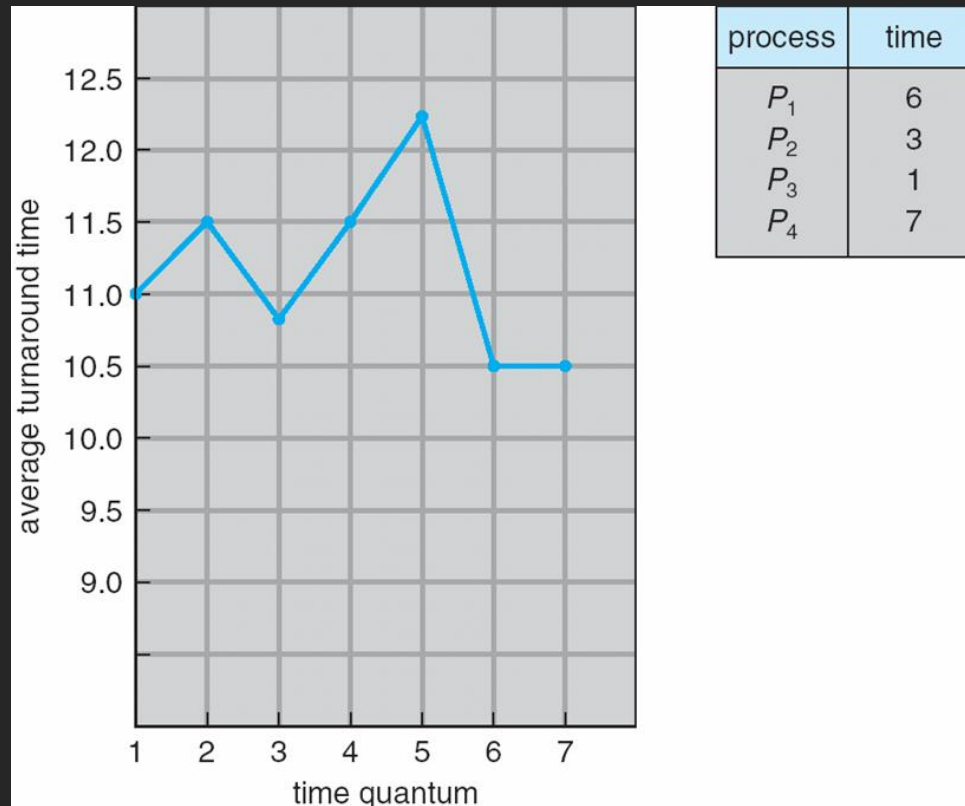
Algoritmos de escalonamento

- Desempenho do RR:
 - Quantum grande: FIFO
 - Quantum pequeno: muito overhead



Algoritmos de escalonamento

- Variação do turnaround em relação a q



Algoritmos de escalonamento

- Algoritmo de Filas Multiníveis:
 - Fila de prontos é dividida em filas distintas:
 - Ex.:
 - Primeiro plano: interativa
 - Segundo plano: batch
 - Cada fila possui um algoritmo de escalonamento:
 - Ex.:
 - Primeiro plano: RR
 - Segundo plano: FCFS
 - É necessário escalonamento entre as filas

Algoritmos de escalonamento

- Escalonamento entre as filas

- Ex. 1:

- Escalonamento de prioridade fixa

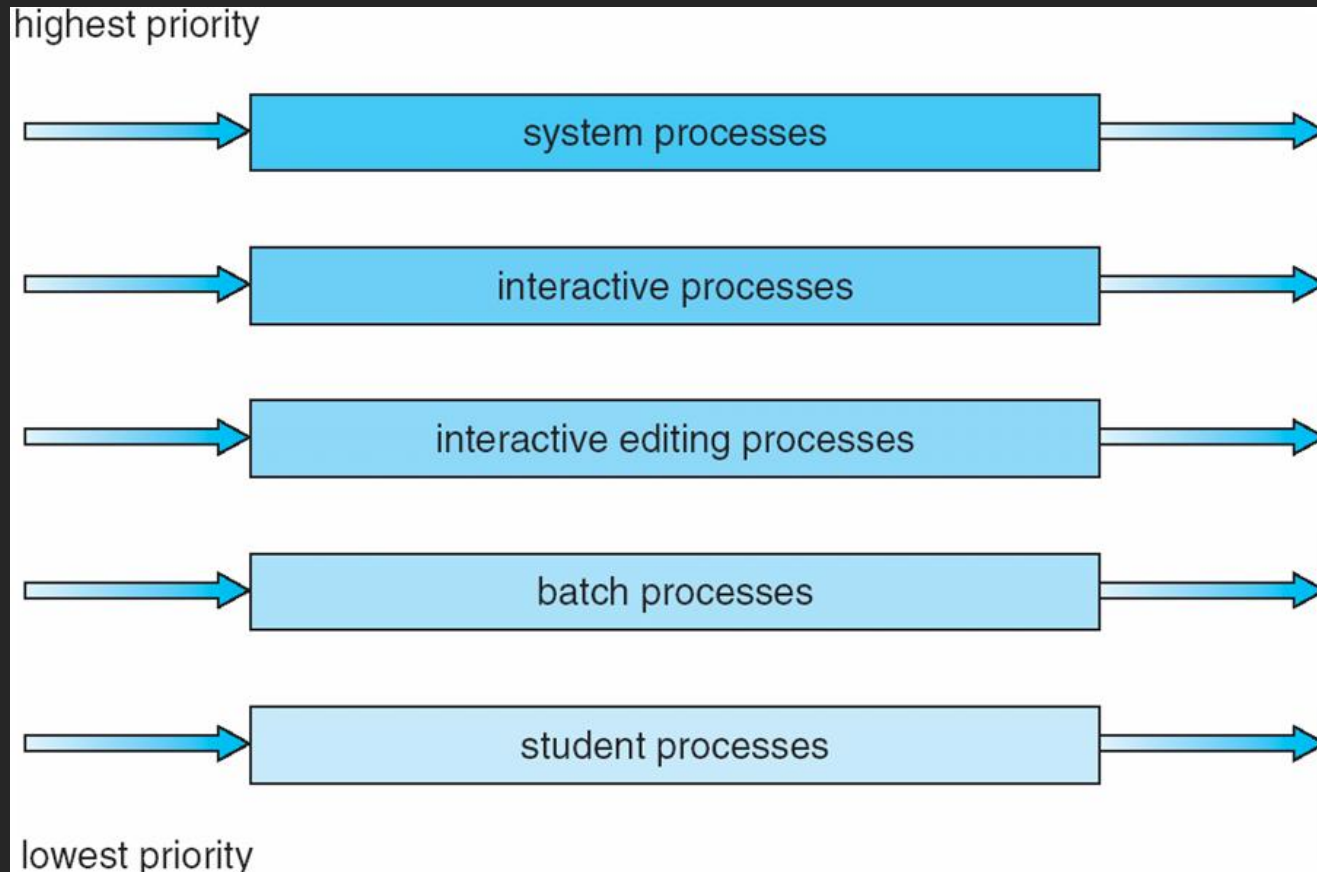
- Serve a todos a partir do primeiro plano e depois do segundo plano
 - Possibilidade de estagnação (starvation)

- Ex. 2:

- Fatia de tempo

- Cada fila recebe uma parte do tempo de CPU
 - Por exemplo, 80% para RR e 20% para FCFS

Algoritmos de escalonamiento



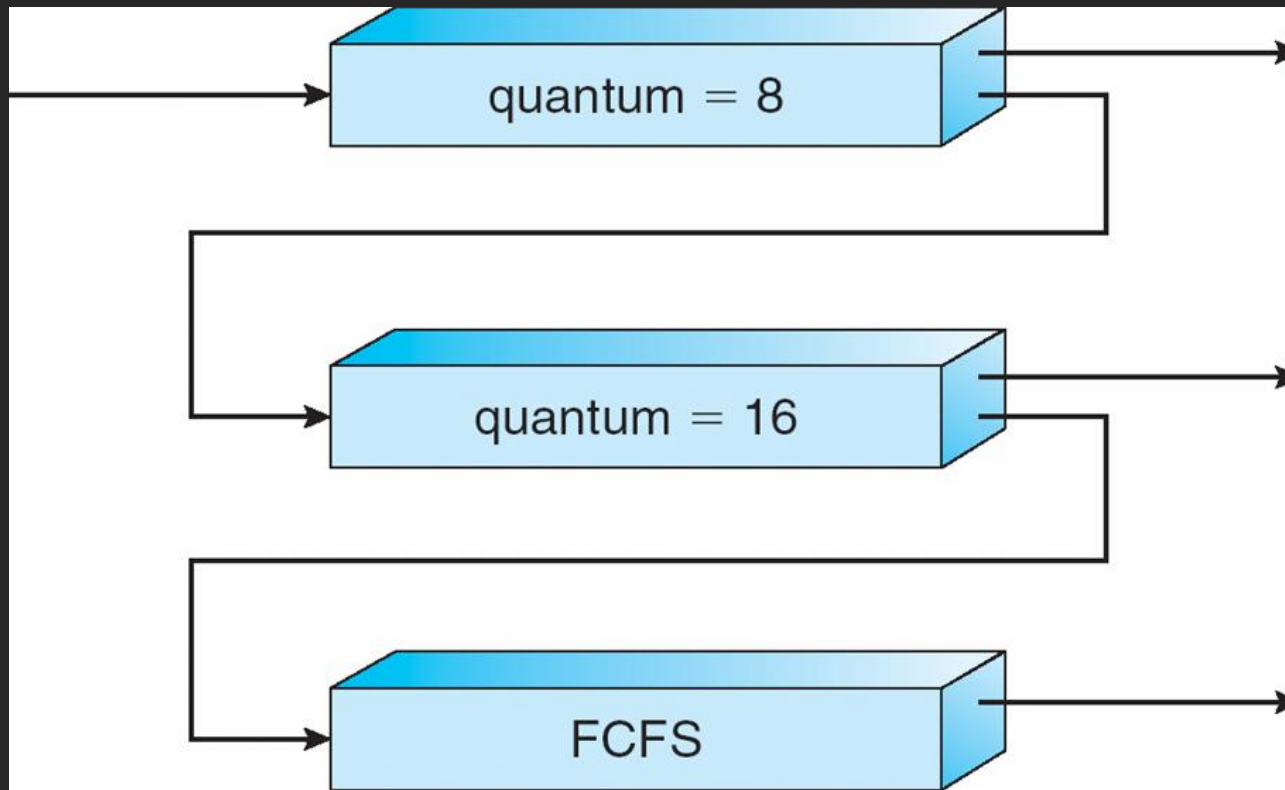
Algoritmos de escalonamento

- Algoritmo de Filas Multiníveis com Retroalimentação:
 - Processo pode ser movido entre as várias filas
 - Envelhecimento pode ser implementado assim
 - Escalonador possui diversos parâmetros
 - Número de filas
 - Algoritmo de escalonamento para cada fila
 - Método para determinar em qual fila processo entra
 - Método para determinar quando elevar um processo
 - Método para determinar quando rebaixar um processo

Algoritmos de escalonamento

- Exemplo:
 - Três filas:
 - Q0: RR com $q = 8\text{ms}$
 - Q1: RR com $q = 16\text{ms}$
 - Q2 : FCFS
 - Escalonamento:
 - Processo novo entra em Q0
 - Em Q0, se pico de CPU não acaba em 8ms, processo vai para Q1
 - Em Q1, se pico de CPU não acaba em 16ms, processo vai para Q2
 - Processo em Q2 permanece nessa fila

Algoritmos de escalonamiento



Escalonamento de threads

- Escalonamento de threads: distinção entre threads de usuário e de kernel
 - Escalonamento PCS (process-contention scope): competição entre threads do mesmo processo
 - Gerenciamento por biblioteca no nível usuário
 - Escalonamento SCS (system-contention scope): competição entre todas as threads no sistema
 - Escalonamento pelo SO
 - Pthreads permite escolher entre PCS e SCS

Escalonamento de threads

```
#include <pthread.h>
#include <stdio.h>
#define NUM_THREADS 5
int main(int argc, char *argv[])
{
    int i;
    pthread_t tid[NUM_THREADS];
    pthread_attr_t attr;
    /* get the default attributes */
    pthread_attr_t attr;
    /* set the scheduling algorithm to PROCESS or SYSTEM */
    pthread_attr_t attr;
    pthread_attr_t attr;
    /* set the scheduling policy - FIFO, RT, or OTHER */
    pthread_attr_t attr;
    pthread_attr_t attr;
    /* create the threads */
    for (i = 0; i < NUM_THREADS; i++)
        pthread_create(&tid[i], &attr, runner, NULL);
```

...

Escalonamento em multiprocessadores

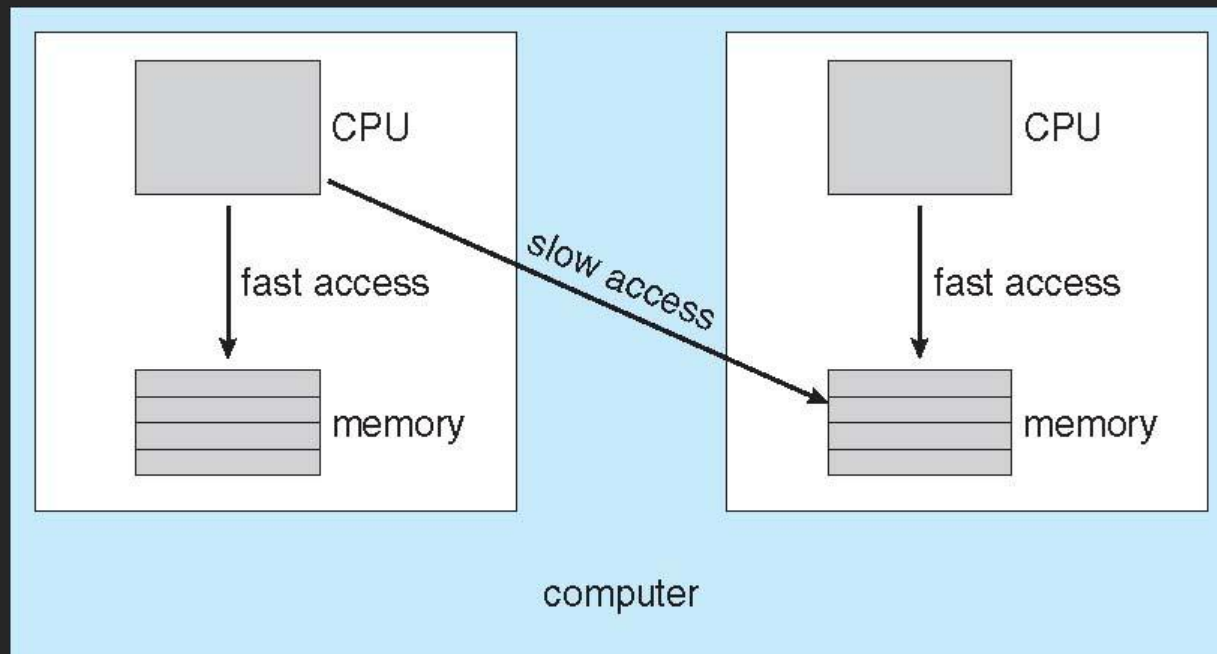
- Múltiplas CPUs: escalonamento mais complexo e com várias possibilidades
 - Multiprocessamento assimétrico: só um processador (mestre) acessa estruturas de dados, demais só executam código de usuário
 - Multiprocessamento simétrico (SMP): dados compartilhados pelos processadores
 - Cada processador é auto-escalonável: fila de prontos pode ser separada por processador ou global
 - Suportado por Windows, Linux, Solaris, Mac OS X

Escalonamento em multiprocessadores

- Afinidade com o processador: processo possui mais afinidade com processador corrente
 - Cache já populada
 - Migrar para outro processador tem custo alto
 - Cache do processador anterior deve ser invalidada
 - Cache do novo processador deve ser repopulada
- Afinidade pode ser soft ou hard:
 - Soft: SO tenta manter no processador, mas não garante
 - Hard: não permite migração
 - Também suportada pelo Linux e outros SOs

Escalonamento em multiprocessadores

- Arquiteturas de memória também pode afetar afinidade
 - Arquitetura NUMA: tempo de acesso à memória varia



Escalonamento em multiprocessadores

- Balanceamento de carga: busca distribuir a carga de trabalho entre todos os processadores do sistema SMP
 - Fila de prontos comum: balanceamento é desnecessário
 - Fila de prontos separada (estratégia de muitos SOs contemporâneos): dois tipos de migração
 - Migração push: tarefa específica periodicamente “empurra” processos da fila do processador sobrecarregado para outro com menor carga
 - Migração pull: processador livre “puxa” processo que está aguardando em um processador sobrecarregado
 - Os dois tipos de migração podem ser executados num mesmo SO
 - Linux: push a cada 200ms e pull quando a fila está vazia

Escalonamento em multiprocessadores

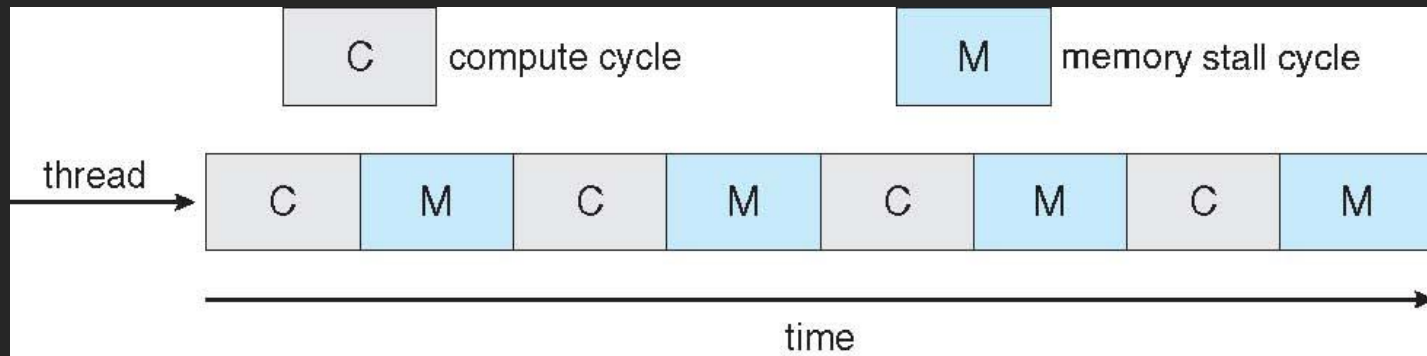
- Balanceamento de carga prejudica os benefícios da afinidade
 - O que é melhor? Balancear? Manter afinidade? Não há consenso!
 - Alguns sistemas:
 - Processador inativo sempre “puxa” processo de processador sobrecarregado
 - Migração ocorre somente se balanceamento excede limiar

Escalonamento em multiprocessadores

- Escalonamento em processadores multicore:
 - Multicore: vários processadores no mesmo chip
 - Mais rápidos e consomem menos energia
 - Grande problema: quando processador acessa memória, espera muito tempo esperando disponibilização do dado (memory stall)
 - Várias razões: ex.: cache miss

Escalonamento em multiprocessadores

- Cenário com 50% ciclos de processamento e 50% stalls de memória

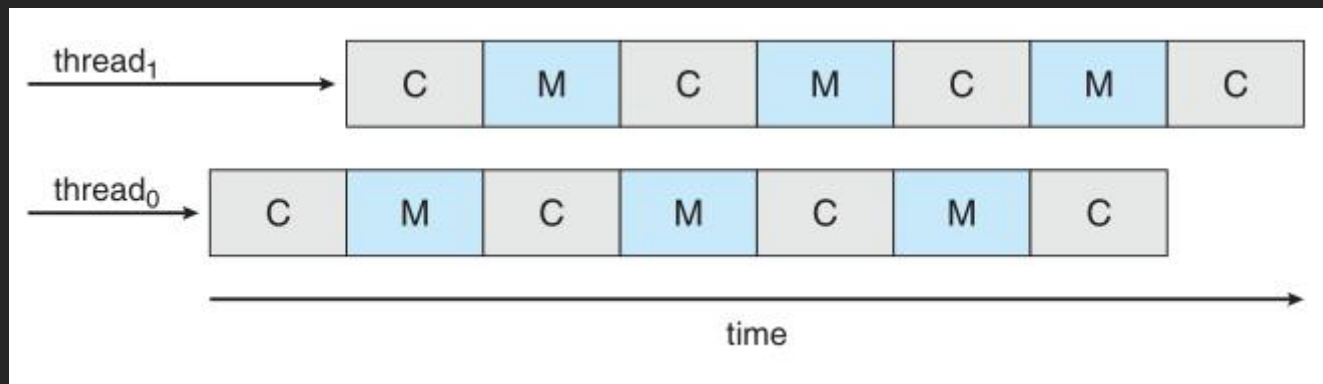


Escalonamento em multiprocessadores

- Solução para agilizar a execução: núcleo multithread
 - Cada núcleo: suporta duas ou mais threads associadas
 - Quando há stall em uma, núcleo executa outra
 - Execução fica intercalada
 - Do ponto de vista do SO: cada thread de hardware é um processador lógico que pode executar threads
 - Dual-thread dual-core: 4 processadores lógicos

Escalonamento em multiprocessadores

- Execução intercalada em um sistema multithread:



Escalonamento em multiprocessadores

- Duas formas de implementar processadores multithread:
 - Coarse-grained multithread: thread executa até evento de longa latência (stall) ocorrer
 - Alto custo para trocar threads: esvaziar pipeline da thread anterior e preencher pipeline da nova thread
 - Fine-grained multithread (interleaved): trocas são feitas em período bem curto (1 ciclo de instrução)
 - Suporte de hardware para troca mais rápida de thread

Escalonamento em multiprocessadores

- Escalonamento em processador multicore multithread: dois níveis
 - Primeiro nível: SO escolhe qual thread de software irá executar em cada thread de hardware (processador lógico)
 - Segundo nível: arquitetura define qual thread de hardware o núcleo do processador irá executar

Exemplos de sistemas operacionais

- Solaris: escalonamento de threads por prioridade
 - Prioridade maior: maior número
 - Cada thread pode pertencer a uma classe dentre: Time sharing (TS), Interactive (IA), Real time (RT), System (SYS), Fair share (FSS), Fixed priority (FP)
 - Dentro de cada classe: diferentes algoritmos/prioridades
 - Classe Time Sharing (TS): usa filas multiníveis com retroalimentação

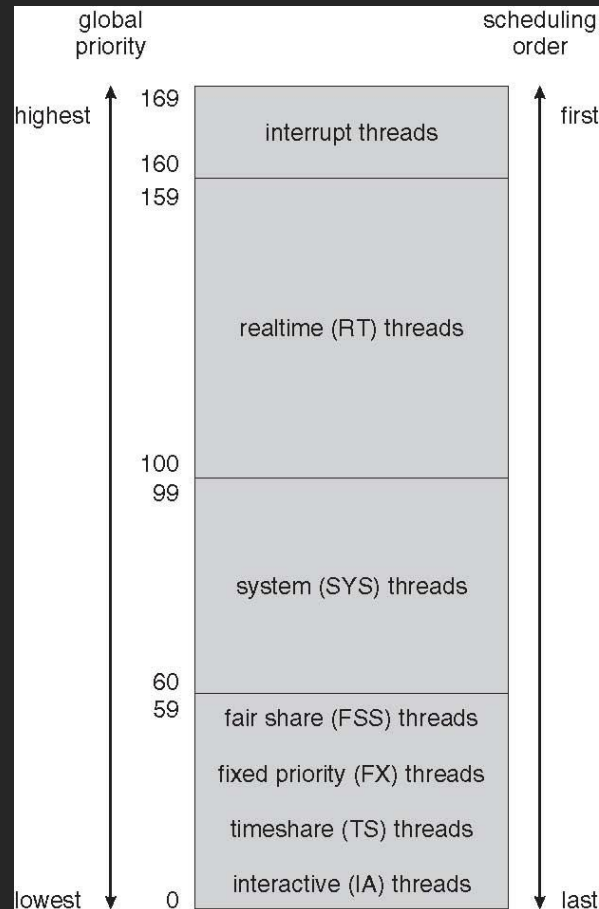
Exemplos de sistemas operacionais

- Tabela de prioridades para escalonar processos TS e IA:

priority	time quantum	time quantum expired	return from sleep
0	200	0	50
5	200	0	50
10	160	0	51
15	160	5	51
20	120	10	52
25	120	15	52
30	80	20	53
35	80	25	54
40	40	30	55
45	40	35	56
50	40	40	58
55	40	45	58
59	20	49	59

Exemplos de sistemas operacionais

- Prioridade entre as classes de escalonamento:



Exemplos de sistemas operacionais

- Windows XP: escalonamento baseado em prioridades preemptivo
 - Thread irá executar até preempção por outra de maior prioridade, ou quantum expirar, ou fazer chamada ao sistema bloqueante (E/S)
 - Classes de prioridade: real-time, high, above normal, normal, below normal, idle priority
 - Prioridade relativa dentro de cada classe: time-critical, highest, above normal, normal, below normal, lowest, idle

Exemplos de sistemas operacionais

- Prioridades do Windows XP:

	real-time	high	above normal	normal	below normal	idle priority
time-critical	31	15	15	15	15	15
highest	26	15	12	10	8	6
above normal	25	14	11	9	7	5
normal	24	13	10	8	6	4
below normal	23	12	9	7	5	3
lowest	22	11	8	6	4	2
idle	16	1	1	1	1	1

Exemplos de sistemas operacionais

- Linux a partir do kernel 2.5: algoritmo $O(1)$
 - Suporte adequado para SMP, afinidade de processador, balanceamento de carga, justiça e suporte para tarefas interativas
 - Algoritmo preemptivo baseado em prioridades:
 - Menor valor: maior prioridade
 - Maior prioridade: maior quantum
 - Tarefas real-time possuem prioridades de 0 a 99 e demais possuem prioridades 100 a 140
 - Prioridades de tarefas (exceto tempo real) podem variar +/- 5

Exemplos de sistemas operacionais

- Prioridades no Linux:

numeric priority	relative priority		time quantum
0	highest	real-time tasks	200 ms
•			
•			
•			
99			
100		other tasks	10 ms
•			
•			
•			
140	lowest		

Exemplos de sistemas operacionais

- Cada CPU possui uma runqueue e faz escalonamento independente
 - Cada runqueue possui dois arrays de prioridade: active e expired
 - Active: tarefas com tempo restante de CPU
 - Expired: tarefas que expirarm tempo
 - Escalonador escolhe tarefa com maior prioridade do array ativo
 - Quando todas as tarefas estão expiradas, array Expired é trocado com Active

Exemplos de sistemas operacionais

- Arrays Active e Expired:



Avaliação de algoritmos

- Modelos de avaliação:
 - Determinístico, de enfileiramento, simulação e implementação
- Modelagem determinística
 1. Pré-definição de tarefas estáticas
 2. Submissão das tarefas para cada algoritmo
 3. Verificação do desempenho, de acordo com critérios de análise

Avaliação de algoritmos

- Modelos de enfileiramento
 - Trabalha com estimativas ou distribuições de picos de CPU e E/S, ao invés de tarefas pré-definidas
 - Possibilita análise sem conhecer exatamente tarefas
- Simulação
 - Envolve a programação de um modelo de sistema de computação
 - Simulador gera dados aleatoriamente: picos, tempos de chegada, etc.
 - De acordo com distribuição de probabilidade
 - Dados podem ser gerados após rastreamento (trace)

Avaliação de algoritmos

- Implementação
 - Modelo de análise mais fiel à realidade
 - Alto custo
 - Codificação/modificação do SO
 - Adaptação/reação dos usuários às mudanças